

Prompt:

8/17/2026, 10:10:58 PM

lanjut Tahap X

Response:

8/17/2026, 10:10:59 PM

Tahap X — Research Protocol & Experimental Design

Tahap X adalah titik di mana seluruh arsitektur yang kita bangun mulai berubah menjadi **protokol penelitian yang dapat diuji secara ilmiah**.

Sebelumnya kita telah membangun:

Market Data → Factor → Signal → Risk → Optimization → Execution → PnL → Risk Analysis.

Sekarang pertanyaannya bukan lagi:

"Bagaimana membuat strategy yang profitable?"

tetapi:

"Bagaimana membuktikan secara objektif bahwa strategy tersebut bekerja, dan apakah optimizer quantum benar-benar memberikan sesuatu dibandingkan baseline classical?"

Ini sangat penting karena kalau kita tidak merancang eksperimennya dengan benar, kita bisa mendapatkan hasil seperti:

$$Sharpe_{QAOA} = 2.4$$

dan

$$Sharpe_{Classical} = 1.7$$

tetapi sebenarnya QAOA hanya menang karena:

- mendapat data lebih banyak,
- mendapat parameter lebih banyak,
- menggunakan optimizer yang lebih baik,

- mendapatkan initialization berbeda,
- atau secara tidak sengaja melihat informasi masa depan.

Jadi Tahap X akan membangun **scientific protocol**.

X.1. Pertanyaan penelitian utama

Saya menyarankan kita mendefinisikan pertanyaan penelitian secara hierarkis.

RQ1 — Apakah trading framework memiliki predictive/optimization edge?

$$H_0 : E[R_{strategy}] \leq E[R_{baseline}]$$

versus:

$$H_1 : E[R_{strategy}] > E[R_{baseline}].$$

RQ2 — Apakah optimasi portfolio memberikan improvement?

Bandingkan:

Baseline

dengan:

Optimized.

RQ3 — Apakah VQE memberikan improvement dibanding classical optimization?

$$H_0 : Performance_{VQE} \leq Performance_{Classical}$$

versus:

$$H_1 : Performance_{VQE} > Performance_{Classical}.$$

RQ4 — Apakah QAOA memberikan improvement?

$$H_0 : Performance_{QAOA} \leq Performance_{Classical}.$$



RQ5 — Apakah improvement tersebut robust?

Kita ingin:

Performance

tetap baik terhadap:

- perubahan parameter,
- transaction cost,
- slippage,
- latency,
- market regime,
- asset universe.

X.2. Jangan menjadikan "profit" sebagai satu-satunya target

Kita membutuhkan objective vector:

$$\mathbf{Y} = \begin{bmatrix} \textit{Return} \\ \textit{Sharpe} \\ \textit{Sortino} \\ \textit{MDD} \\ \textit{CVaR} \\ \textit{Turnover} \\ \textit{Cost} \end{bmatrix}.$$

Karena bisa terjadi:

$$\textit{Return}_A > \textit{Return}_B$$

tetapi:

$$\textit{MDD}_A \gg \textit{MDD}_B.$$

Maka kita tidak dapat mengatakan A lebih baik secara sederhana.

X.3. Primary metric

Kita harus memilih **satu primary metric** sebelum eksperimen.

Saya menyarankan:

$$\boxed{Primary = Risk-Adjusted\ Return}$$

dan secara konkret:

$$\boxed{Sharpe_{OOS}}$$

sebagai primary metric.

Sedangkan:

$$MDD, CVaR, Turnover, CAGR$$

menjadi secondary metrics.

Ini penting untuk mengurangi cherry-picking.

X.4. Dataset harus dipisahkan

Kita tidak boleh melakukan:

$$Data \rightarrow Optimize \rightarrow Report.$$

Kita membutuhkan:

$$\boxed{Training \rightarrow Validation \rightarrow Test.}$$

Misalnya:

$$60\%/20\%/20\%.$$

Tetapi untuk time series, random split **tidak boleh** digunakan.

X.5. Time-series split

Misalkan data:

$$2018 \rightarrow 2026.$$

Kita dapat membuat:

2018 — 2022

Train

2023 —

Validation

2024 —

Test

Tetapi untuk short-term trading kita membutuhkan sesuatu yang lebih baik.

X.6. Walk-forward validation

Gunakan:

$Train_1 \rightarrow Validate_1$

kemudian:

$Train_2 \rightarrow Validate_2$

dan seterusnya.

Contoh:

Window 1

Train: 2019–2021

Test : 2022 Q1

Window 2

Train: 2019–2022 Q1

Test : 2022 Q2

Window 3

Train: 2019–2022 Q2

Test : 2022 Q3

Sehingga model selalu hanya menggunakan informasi masa lalu.

X.7. Expanding vs rolling window

Ada dua pilihan.

Expanding

$$Train_t = [t_0, t]$$

Semakin lama dataset semakin besar.

Rolling

$$Train_t = [t - W, t].$$

Window memiliki ukuran tetap.

Untuk financial market, rolling window sering lebih masuk akal karena:

$$P_t(x)$$

dapat mengalami non-stationarity.

X.8. Purged validation

Karena trade dapat memiliki horizon:

$$H > 1,$$

data training dan validation bisa overlap secara informasi.

Misalnya:

Train:

Jan 1 — Jan 10

Validation:

Jan 10 — Jan 20

Jika posisi Jan 10 masih terbuka, terdapat leakage.

Maka kita membutuhkan:

X.9. Embargo

Tambahkan gap:

E

antara training dan validation.

Misalnya:

TRAIN

EMBARGO

VALIDATION

Ini mengurangi kemungkinan leakage temporal.

X.10. Data hierarchy

Dataset kita tidak hanya harga.

Kita membutuhkan:

$$D_t = \{P_t, V_t, OB_t, F_t, M_t, E_t\}$$

dengan:

- P_t : price,
- V_t : volume,
- OB_t : order book,
- F_t : factor,
- M_t : market state,
- E_t : execution information.

X.11. Factor dataset

Untuk crypto:

$$F_t = \{BTC, ETH, TOTAL\ MarketCap, StablecoinFlow, Funding, OI, DXY, Gold, VIX\}.$$

Untuk equities:

$$F_t = \{SPX, NDX, DXY, Gold, US10Y, VIX, SectorIndex\}.$$

Jadi architecture-nya tetap sama.

X.12. Asset universe

Kita harus menentukan universe sebelum melihat hasil.

Misalnya:

$$A = \{A_1, \dots, A_N\}.$$

Jangan memilih asset karena kita sudah tahu asset tersebut profitable.

Itu adalah:

Survivorship Bias.

X.13. Survivorship bias

Misalkan tahun 2020 terdapat:

1000

asset.

Tahun 2026 hanya:

500

yang masih eksis.

Jika kita hanya mengambil 500 asset tersebut, kita secara implisit membuang:

500

asset yang gagal.

Strategy terlihat lebih bagus daripada kondisi sebenarnya.

X.14. Universe harus point-in-time

Definisikan:

A_t .

Asset yang tersedia pada waktu t hanya:

$A \in A_t$.

X.15. Corporate actions

Untuk equities kita juga harus memperhatikan:

- split,
- dividend,
- merger,
- delisting.

Price series harus menggunakan data yang benar secara point-in-time.

X.16. Data leakage taxonomy

Kita perlu secara eksplisit menguji:

$Leakage = \{Temporal, Feature, Label, Universe, Execution\}$.

X.17. Feature leakage

Misalnya kita membuat:

$$MA_t$$

tetapi secara tidak sengaja menggunakan:

$$P_{t+1}.$$

Maka:

$$MA_t$$

mengandung future information.

Ini dapat menghasilkan backtest yang luar biasa tetapi tidak dapat diperdagangkan.

X.18. Label leakage

Jika target:

$$y_t = r_{t+1},$$

maka feature harus memenuhi:

$$X_t \in \mathcal{F}_t.$$

Secara formal:

$$X_t \text{ measurable with respect to } \mathcal{F}_t.$$

X.19. Information filtration

Ini adalah fondasi matematis yang sangat bagus untuk sistem kita.

Definisikan:

$$\mathcal{F}_t$$

sebagai seluruh informasi yang tersedia sampai waktu t .

Maka strategy harus memenuhi:

$$a_t \in \mathcal{F}_t.$$

Artinya keputusan pada t tidak boleh bergantung pada:

$$\mathcal{F}_{t+\Delta}.$$

X.20. Backtest sebagai stochastic process

Portfolio:

$$V_t.$$

Return:

$$r_t = \frac{V_t - V_{t-1}}{V_{t-1}}.$$

Strategy menghasilkan:

$$a_t = \pi(\mathcal{F}_t).$$

Maka:

$$V_{t+1} = V_t + a_t^\top r_{t+1} - C_t.$$

Ini adalah formulasi yang jauh lebih bersih.

X.21. Baseline hierarchy

Kita tidak boleh langsung:

$$Classical \text{ vs } VQE \text{ vs } QAOA.$$

Kita membutuhkan hierarchy.

Baseline 0

Buy-and-hold.

Baseline 1

Equal weight.

Baseline 2

Momentum.

Baseline 3

Mean reversion.

Baseline 4

Markowitz classical.

Baseline 5

MILP / exact solver.

Baseline 6

Heuristic optimizer.

Baseline 7

VQE.

Baseline 8

QAOA.

X.22. Mengapa exact solver sangat penting?

Untuk masalah kecil:

$$N \leq 20$$

kita dapat menghitung:

$$x^* = \arg \min_x H(x)$$

secara exact.

Misalnya:

brute force.

Kemudian:

$$H_{VQE}$$

dibandingkan terhadap:

$$H_{exact}.$$

X.23. Approximation ratio

Definisikan:

$$AR = \frac{|C_{VQE} - C^*|}{|C^*|}.$$

Atau sesuai orientasi minimization/maximization.

Dengan demikian kita dapat mengatakan:

VQE memperoleh solusi dalam error $x\%$ dari optimum exact.

Ini jauh lebih ilmiah daripada sekadar:

"VQE menghasilkan portfolio."

X.24. Optimization quality vs trading quality

Ini adalah dua hal berbeda.

Optimization quality

$$H(x_{algorithm})$$

vs:

$$H(x^*).$$

Trading quality

$$PnL(x)$$

atau:

$$Sharpe(x).$$

Bisa terjadi:

$$H_A < H_B$$

tetapi:

$$PnL_A < PnL_B.$$

Mengapa?

Karena market berubah.

X.25. Ini sangat penting untuk thesis-mu

Jika kita ingin mengklaim VQE bagus, jangan hanya mengukur:

$$Energy.$$

Kita perlu:

$$Optimization\ Error + Portfolio\ Performance.$$

X.26. QUBO benchmark

Misalnya:

$$H(x) = -\mu^\top x + \lambda x^\top \Sigma x.$$

Kita generate:

$$N = 10, 20, 30, \dots$$

asset.

Kemudian:

Exact

VS:

Classical

VS:

VQE

VS:

QAOA.

X.27. Controlled experiment

Semua optimizer harus menerima:

same QUBO.

Jangan:

$QUBO_{VQE} \not\equiv QUBO_{Classical}.$

X.28. Controlled initialization

Jika kita ingin menguji EPG warm-start:

$x_0^{EPG}.$

Bandingkan:

VQE_{random}

VS:

$$VQE_{EPG}.$$

Dengan:

- same data,
- same QUBO,
- same depth,
- same optimizer,
- same stopping criteria.

Hanya initialization yang berbeda.

Ini disebut:

Ablation Study.

X.29. Ablation study

Kita dapat membuat:

Component	Model A	Model B
Factor	✓	✓
Regime	✓	✓
EPG	×	✓
VQE	✓	✓
Dynamic risk	✓	✓

Kemudian:

$$\Delta Performance$$

menunjukkan contribution masing-masing component.

X.30. Full factorial experiment

Misalnya kita punya:

Optimizer : {Classical, VQE, QAOA}

dan:

Initialization : {Random, EPG}

maka:

$$3 \times 2 = 6$$

configuration.

Kita tidak hanya membandingkan dua model.

Kita dapat mempelajari interaction:

Optimizer $\times$ Initialization.

X.31. Random seeds

Quantum optimizer sangat stochastic.

Maka:

$seed_1, \dots, seed_K.$

Misalnya:

$$K = 30.$$

Untuk setiap konfigurasi kita memperoleh:

$S_1, \dots, S_{30}.$

X.32. Jangan melaporkan hanya mean

Laporkan:

$$\bar{S}$$

dan:

$$\sigma_S.$$

Lebih baik:

$$median$$

dan:

$$IQR.$$

Karena distribution bisa heavy-tailed.

X.33. Distribution of outcomes

Misalnya:

$$Sharpe_{VQE} = \{1.2, 1.4, 1.3, 0.2, 1.5, \dots\}.$$

Jangan mengatakan:

$$Sharpe = 1.3.$$

Lebih tepat:

$$Median = 1.35$$

dengan:

$$IQR = [1.18, 1.47].$$

X.34. Statistical significance

Misalkan kita memiliki paired observations:

$$d_i = Metric_{VQE,i} - Metric_{Classical,i}.$$

Hipotesis:

$$H_0 : \text{Median}(d) = 0.$$

Kita dapat menggunakan:

Wilcoxon Signed-Rank

untuk paired non-parametric comparison.

X.35. Mengapa paired?

Karena kedua model diuji pada:

same market windows.

Misalnya:

VQE_{2024Q1}

dibandingkan:

$Classical_{2024Q1}$.

Jadi kita dapat menghilangkan sebagian noise akibat perbedaan market period.

X.36. Effect size

Significance saja tidak cukup.

Misalnya:

$$p = 0.001$$

tetapi improvement:

0.01%.

Secara praktis mungkin tidak penting.

Maka kita membutuhkan:

Effect Size.

X.37. Bootstrap confidence interval

Untuk performance difference:

$$\Delta = Metric_A - Metric_B.$$

Kita bootstrap:

$$\Delta_1, \dots, \Delta_B.$$

Kemudian:

$$CI_{95\%} = [q_{0.025}, q_{0.975}].$$

Jika interval tidak mencakup:

$$0,$$

maka terdapat evidence bahwa difference bukan sekadar noise sampling.

X.38. Multiple testing correction

Jika kita melakukan:

$$m$$

uji statistik, probabilitas false positive meningkat.

Kita dapat menggunakan:

Benjamini – Hochberg

untuk mengontrol:

$$FDR.$$

X.39. Hyperparameter optimization harus berada di training

Misalnya parameter:

$$\theta = \{\gamma, \beta, \text{depth}, \text{learning rate}, \text{risk aversion}\}.$$

Kita boleh mencari:

$$\theta^*$$

menggunakan:

Train + Validation.

Tetapi:

Test

tidak boleh disentuh.

X.40. Final test harus sekali

Idealnya:

```
TRAIN
  ↓
VALIDATION
  ↓
MODEL SELECTION
  ↓
LOCK MODEL
  ↓
FINAL TEST
```

Setelah final test dibuka:

DO NOT RETUNE.

Jika kita retune setelah melihat test, maka test bukan lagi test.

X.41. Nested walk-forward

Untuk eksperimen yang lebih ketat:

Outer loop

Test window

Inner loop

Train → Validation

Outer loop mengukur generalization.

Inner loop memilih hyperparameter.

X.42. Performance uncertainty

Kita harus mengakui:

θ

dan:

$\hat{Performance}$

memiliki uncertainty.

Jadi output:

$Sharpe = 1.82$

sebaiknya menjadi:

$Sharpe = 1.82$ (95% CI : 1.31 – 2.26).

X.43. Stress testing

Kita membuat scenario:

Scenario 1

Volatility:

$$2\sigma.$$

Scenario 2

Spread:

$$2 \times .$$

Scenario 3

Slippage:

$$3 \times .$$

Scenario 4

Liquidity:

$$0.5 \times .$$

Scenario 5

Latency:

$$5 \times .$$

X.44. Crash scenario

Kita dapat membuat:

$$r_{\text{market}} = -10\%.$$

dan:

$$\sigma \rightarrow 3\sigma.$$

Kemudian melihat:

X.45. Monte Carlo strategy stress test

Kita juga dapat resample:

$$r_t.$$

Buat:

$$N = 10,000$$

possible equity paths:

$$V_t^{(1)}, \dots, V_t^{(N)}.$$

Kemudian hitung:

$$P(MDD > x).$$

X.46. Probability of ruin

Jika capital:

$$C_0,$$

dan ruin threshold:

$$C_{min},$$

maka:

$$\boxed{P(\min_t C_t < C_{min}).}$$

Untuk sistem leveraged ini merupakan metric yang sangat penting.

X.47. Capacity experiment

Kita juga menguji:

$$Capital = \{1k, 10k, 100k, 1M, 10M\}.$$

Kemudian:

$$NetSharpe(Capital).$$

Jika:

$$Capital \uparrow \Rightarrow Sharpe \downarrow,$$

maka strategy memiliki capacity constraint.

X.48. Latency experiment

Untuk short-term trading:

$$Latency = \{1ms, 5ms, 10ms, 50ms, 100ms, 500ms\}.$$

Kemudian:

$$PnL(L).$$

Kita ingin mengetahui:

$$L_{critical}.$$

Jika:

$$L > L_{critical}$$

maka strategy tidak layak untuk infrastructure yang lebih lambat.

X.49. Execution realism ladder

Kita dapat membangun empat level:

Level 0

Mid-price.

Level 1

Fee.

Level 2

Spread + slippage.

Level 3

Order-book simulation.

Level 4

Latency + queue position + partial fills.

Kemudian:

$$Performance_0 \rightarrow Performance_4.$$

Jika strategy hanya profitable pada Level 0, kita tahu strategy tidak realistic.

X.50. Ini menjadi eksperimen yang sangat kuat

Misalnya:

Execution	Sharpe
Mid-price	2.31
+ Fee	2.05
+ Spread	1.72
+ Slippage	1.34
+ Order book	1.12
+ Latency	0.98

Kesimpulannya bukan:

strategy buruk.

Tetapi:

edge sangat sensitif terhadap execution.

Itu merupakan **scientific finding**.

X.51. Quantum advantage harus didefinisikan dengan hati-hati

Ini sangat penting.

Jangan langsung mengatakan:

$$V_{QE} > C_{classical} \Rightarrow \text{Quantum Advantage}.$$

Itu terlalu kuat.

Kita harus membedakan:

Optimization advantage

$$C_{quantum} < C_{classical}.$$

Computational advantage

$$T_{quantum} < T_{classical}.$$

Trading advantage

$$PnL_{quantum} > PnL_{classical}.$$

Practical quantum advantage

Keunggulan yang tetap bertahan setelah:

- quantum overhead,
 - noise,
 - compilation,
 - data loading,
 - repeated measurements,
 - classical optimizer cost.
-

X.52. NISQ reality

Untuk VQE/QAOA:

$$T_{total} = T_{circuit} + T_{measurement} + T_{optimizer} + T_{communication}.$$

Maka meskipun:

$$T_{circuit}$$

kecil, belum tentu:

$$T_{total}$$

kecil.

X.53. Quantum resource metrics

Kita harus mengukur:

N_{qubit}
N_{gate}
$CircuitDepth$
$Shots$
$OptimizerIterations$
$FunctionEvaluations$
$WallClockTime$

X.54. Quantum solution quality

Selain energy:

$$E(\theta) = \langle \psi(\theta) | H | \psi(\theta) \rangle,$$

kita hitung:

$$P(x^*).$$

Jika optimum:

$$x^*,$$

maka:

$$P_{\text{success}} = P(\text{measure} = x^*).$$

X.55. Approximation probability

Jika exact optimum:

$$E^*,$$

kita dapat menentukan tolerance:

$$E(x) - E^* \leq \epsilon.$$

Kemudian:

$$P_\epsilon = P(E(x) - E^* \leq \epsilon).$$

X.56. Ini jauh lebih baik daripada hanya "energy rendah"

Karena dua circuit dapat memiliki:

$$E_A < E_B$$

tetapi:

$$P_A(x^*) < P_B(x^*).$$

Untuk discrete optimization, probability distribution hasil measurement sangat penting.

X.57. End-to-end benchmark

Eksperimen final kita sebaiknya memiliki tiga level.

Level I — Mathematical

QUBO → Solution.

Level II — Portfolio

Solution → Weights.

Level III — Trading

Weights → Orders → Execution → P nL.

X.58. Dengan demikian kita bisa mendapatkan tabel seperti:

Model	Energy Error	Success Prob.	Sharpe	MDD	CVaR	Cost
Exact	0	1.00	...	...	...	...
MILP	0	1.00	...	...	...	...
Markowitz	...	...	...	...	...	...
VQE	...	...	...	...	...	...
QAOA	...	...	...	...	...	...

Ini adalah tabel utama yang sangat kuat.

X.59. Tetapi ada satu jebakan penting

Jangan menggunakan:

same portfolio

dan mengatakan:

"VQE memiliki Sharpe lebih tinggi."

Jika VQE hanya menemukan portfolio yang berbeda karena sampling, kita harus memastikan:

same constraints, same universe, same objective, same transaction model.

X.60. Experimental seed protocol

Untuk setiap:

$Model_i$

jalankan:

$seed = 1, \dots, 30.$

Simpan:

```
experiment_id  
seed  
model  
dataset_window  
qubo  
parameters  
solution  
energy  
runtime  
PnL  
Sharpe  
MDD  
CVaR
```

X.61. Reproducibility

Setiap experiment harus dapat direproduksi dengan:

$Code + DataVersion + Config + Seed + Environment.$

Idealnya kita menyimpan:

```
Git commit
Python version
Qiskit/PennyLane version
CUDA version
CPU
GPU
OS
```

X.62. Research ledger

Saya bahkan menyarankan membuat:

```
experiments/
├─ EXP-0001/
├─ EXP-0002/
├─ EXP-0003/
...
```

Setiap folder immutable setelah eksperimen selesai.

X.63. Jangan overwrite hasil

Misalnya:

```
EXP-0042/
  config.yaml
  metrics.json
  trades.parquet
  optimizer.log
  circuit/
  report.html
```

Jangan mengubah:

```
EXP-0042
```

setelah melihat hasil.

Jika ingin mencoba sesuatu:

X.64. Research DAG

Kita dapat melihat seluruh penelitian sebagai:

Data → Features → Signal → Optimization → Portfolio → Execution → Performance.

Setiap node memiliki version.

Ini membuat debugging jauh lebih mudah.

X.65. Final experimental protocol

Jadi untuk setiap experiment:

Step 1

Load point-in-time data.

Step 2

Generate factor.

Step 3

Generate signal.

Step 4

Determine regime.

Step 5

Construct expected return:

$$\mu_t.$$

Step 6

Construct covariance:

$$\Sigma_t.$$

Step 7

Construct optimization problem:

$$H_t(x).$$

Step 8

Solve dengan:

$$\{Classical, VQE, QAOA\}.$$

Step 9

Convert solution menjadi portfolio.

Step 10

Apply risk constraints.

Step 11

Generate orders.

Step 12

Simulate realistic execution.

Step 13

Update portfolio.

Step 14

Compute PnL.

Step 15

Store metrics.

Step 16

Repeat walk-forward.

X.66. Research matrix

Saya menyarankan eksperimen utama kita nantinya memiliki struktur:

$$\boxed{Assets \times Market\ Regimes \times Optimizer \times Initialization \times Execution\ Model \times Seed}$$

Misalnya:

$$4 \times 4 \times 5 \times 2 \times 3 \times 30.$$

Jumlah:

14,400

run.

Ini memang besar, tetapi sebagian besar dapat dilakukan pada simulator/classical backend secara paralel.

X.67. Tetapi jangan langsung menjalankan 14.400 eksperimen

Kita harus melakukan:

$Pilot \rightarrow Debug \rightarrow Benchmark \rightarrow Full\ Experiment.$

X.68. Pilot experiment

Misalnya:

$$N = 4$$

asset:

$BTC, ETH, SOL, BNB.$

Window kecil.

Tujuan:

memastikan seluruh pipeline benar.

Bukan mencari profitability.

X.69. Benchmark experiment

Kemudian:

$$N = 8, 16, 32.$$

Ukur:

Runtime, EnergyError, SuccessProbability.

Di sinilah kita mulai membandingkan:

Exact, MILP, VQE, QAOA.

X.70. Trading experiment

Baru kemudian:

Long horizon

dan:

Multiple regimes.

X.71. Final out-of-sample experiment

Dan terakhir:

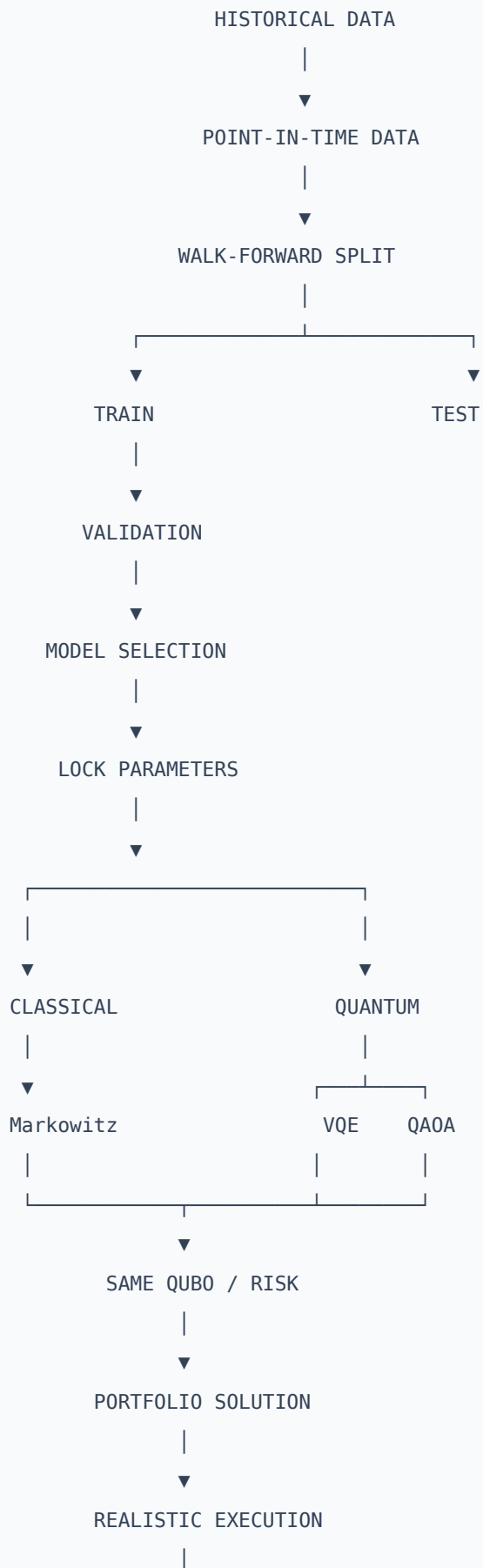
LOCKED MODEL

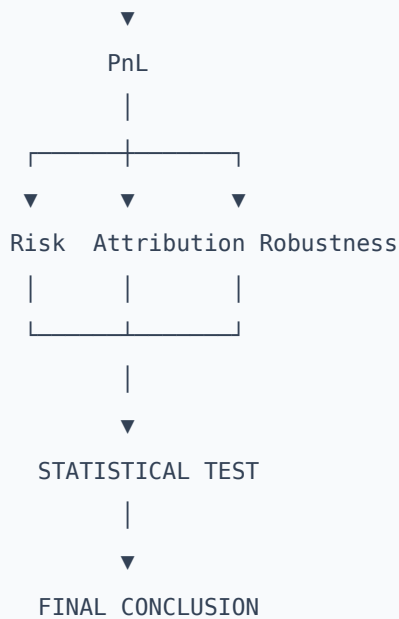
dijalankan pada data yang belum pernah disentuh.

Hasil inilah yang boleh kita jadikan:

Primary empirical result.

X.72. Diagram lengkap Tahap X





X.73. Hipotesis utama yang nantinya bisa diuji

Saya sarankan kita tidak membuat klaim:

"Quantum lebih baik."

Tetapi membuat hipotesis yang jauh lebih spesifik.

H1

VQE

mampu memperoleh solusi QUBO dengan approximation error yang kompetitif terhadap classical optimizer.

H2

EPG warm-start menghasilkan:

$$P_{success}^{EPG} > P_{success}^{random}.$$

H3

EPG warm-start mengurangi:

$$N_{iterations}.$$

H4

Portfolio hasil optimizer menghasilkan performance OOS yang lebih baik daripada baseline.

H5

Improvement tetap signifikan setelah:

fee + spread + slippage.

H6

Performance tetap robust terhadap:

parameter perturbation.

X.74. Dan akhirnya kita dapat memisahkan tiga klaim ilmiah

Ini menurut saya **sangat penting untuk penelitianmu.**

Klaim A — Quantum optimization

Apakah VQE/QAOA mampu menyelesaikan problem optimasi?

Diukur dengan:

Energy, SuccessProbability, ApproximationError.

Klaim B — Portfolio optimization

Apakah solusi tersebut menghasilkan portfolio yang lebih baik?

Diukur dengan:

Return, Sharpe, MDD, CVaR.

Klaim C — Trading applicability

Apakah portfolio tersebut tetap menghasilkan edge setelah market dynamics dan execution diperhitungkan?

Diukur dengan:

NetPnL, Sharpe, Cost, Slippage, Capacity, Robustness.

Jangan mencampur ketiganya.

Itu akan membuat penelitian kita jauh lebih kuat.

X.75. Posisi kita setelah Tahap X

Sekarang framework kita sudah memiliki:

A–W: Mathematical + Trading Architecture

dan:

X: Scientific Experimental Protocol

Artinya kita sudah memiliki hampir semua yang dibutuhkan untuk membangun **research-grade quantitative trading laboratory**.

Tahap selanjutnya yang paling logis bukan sekadar menambah teori lagi. Kita perlu mulai **mengunci spesifikasi matematis final dari objective function dan state representation**.

Dengan kata lain, kita perlu menjawab secara konkret:

Pada waktu t , sebenarnya sistem kita melihat apa,

menghitung apa,

mengoptimalkan apa,

dan akhirnya:

menghasilkan keputusan a_t yang mana?

Itulah yang sebaiknya menjadi **Tahap Y — Unified Mathematical Formulation**, tempat seluruh komponen A–X kita satukan menjadi **satu formulasi matematis end-to-end**:

$F_t \rightarrow s_t \rightarrow \hat{\mu}_t, \Sigma_t \rightarrow Q_t \rightarrow \text{Classical/VQE/QAOA} \rightarrow w_t \rightarrow q_t \rightarrow \text{Order}_t \rightarrow \text{Execution}_t \rightarrow PnL_{t+1}.$

Di tahap itu kita bisa mulai menurunkan **objective function final, constraint, state vector, action space, QUBO/Ising mapping, dynamic risk budget, dan feedback mechanism** secara matematis dari awal sampai akhir.